RK818 Developer Guide

ID: RK-KF-YF-069

Release Version: V1.0.1

Release Date: 2022-05-30

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2022. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, No.89, Software Boulevard, TongPan Rd, Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily introduces the various sub-modules of the RK818, covering related concepts, functions, DTS configuration, and analysis and troubleshooting of common issues.

Product Version

Chip Models	Kernel Version
RK818	3.10, 4.4, 4.19

Intended Audience

This document (this guide) is primarily intended for the following engineers:

- Technical Support Engineers
- Software Development Engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Zhang Qing	2019-11-25	Initial version
V1.0.1	Huang Ying	2022-05-30	Format modification

Table of Contents

RK818 Developer Guide
1. Basics
1.1 Overview
1.2 Functions
1.3 Chip Pin Functions
1.4 Key Concepts
1.5 Power-On Conditions and Sequence
2. Configuration
2.1 Driver and menuconfig
2.2 DTS Configuration
2.3 Function Interfaces
3. Debug
3.1 Kernel 3.10
3.2 Kernel 4.4
3.3 Kernel 4.19

1. Basics

1.1 Overview

The RK818 is a high-performance PMIC. It integrates four high-current DCDC converters, one high-current BOOST converter, nine LDOs, one SWITCH, one HDMI5V output, one OTG output, one RTC, and adjustable power-on sequencing. Additionally, it incorporates switched-mode charging, intelligent power path management, and a coulomb counter.

Power rails in the system are generally categorized into two types: DCDC and LDO. The general characteristics of these two power types are as follows (please search for detailed information independently):

1. DCDC: Offers high efficiency when there is a large difference between input and output voltage, but suffers from relatively large ripple and higher cost. Therefore, it is used for applications with large voltage differentials and high current loads. It generally operates in two modes. PWM mode: Good ripple and transient response, but lower efficiency; PFM mode: High efficiency, but poor load capability.
2. LDO: Exhibits low efficiency and low cost when there is a large difference between input and output voltage. To improve LDO conversion efficiency, the system implements relevant optimizations. For example, if the LDO output voltage is 1.1V, its input voltage can be supplied by a DCDC rail such as VCCIO_3.3V to enhance efficiency. Therefore, if the circuit design permits, LDOs should be connected to DCDC output rails as much as possible, but power-on sequencing must be considered.

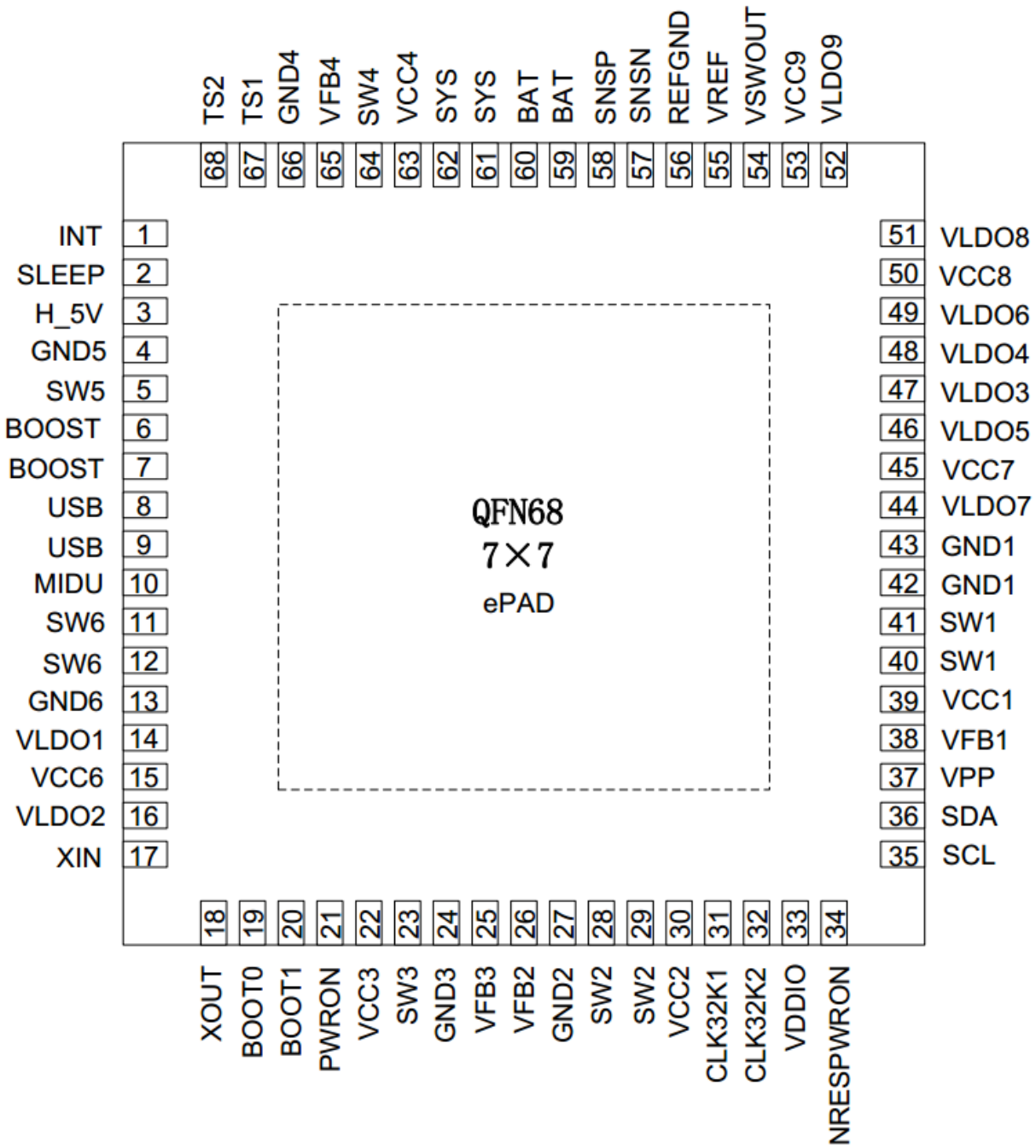
1.2 Functions

From the user's perspective, the functions of the RK818 can be summarized into four parts:

1. Regulator function: Controls the power status of each DCDC and LDO rail;
2. RTC function: Provides clock timing, alarm, and related functions;
3. CLK function: Provides two 32.768kHz clock outputs; one is always-on and non-controllable, while the other is software-controllable.
4. Charging and fuel gauge functions are not detailed in this document. For comprehensive information, refer to "Rockchip_RK818_RK816_Developer_Guide_Fuel_Gauge_CN".

1.3 Chip Pin Functions

QFN68 7mm x 7mm, pitch0.35mm



In the following description, special attention should be paid to the SLEEP and INT pins:

管脚序号	名称	描述
1	INT	Interrupt request pin. Active low.
2	SLEEP	Input pin for switching state between sleep and non-sleep state.
3	H_5V	5v supply output for HDMI
4	GND5	Power ground

5	SW5	Switch output
6,7	BOOST	BOOST output
8,9	USB	Power input from USB
10	MIDU	Middle point of USB power supply
11,12	SW6	Switch output
13	GND6	Power ground
14	VLDO1	LDO1 output
15	VCC6	Power supply for LDO
16	VLDO2	LDO2 output
17	XIN	32.768KHz crystal oscillator input
18	XOUT	32.768KHz crystal oscillator output
19	BOOT0	Boot sequence selection, low bit
20	BOOT1	Boot sequence selection, high bit
21	PWRON	Power on or power off enable pin, active low, internal 100K pull high to power supply
22	VCC3	Power supply for DCDC3
23	SW3	Switch output of DCDC3
24	GND3	Power ground for DCDC3
25	VFB3	feedback voltage for DCDC3
26	VFB2	DCDC2 output voltage feedback input
27	GND2	Power ground for DCDC2
28,29	SW2	Switch output of DCDC2
30	VCC2	Power supply for DCDC2
31	CLK32K1	32.768K clock1 output, open drain,
32	CLK32K2	32.768K clock2 output, open drain,
33	VDDIO	Power supply for IO
34	NRESPWON	Reset pin after power on, active low
35	SCL	Clock input of I2C
36	SDA	Data input/output of I2C
37	VPP	Power supply for testing, floating in the application
38	VFB1	DCDC1 output voltage feedback input
39	VCC1	Power supply for DCDC1
40,41	SW1	Switch output of DCDC1
42,43	GND1	Power ground for DCDC1
44	VLDO7	LDO7 output
45	VCC7	Power supply for LDO
46	VLDO5	LDO5 output

47	VLDO3	LDO3 output
48	VLDO4	LDO4 output
49	VLDO6	LDO6 output
50	VCC8	Power supply for switch
51	VLDO8	LDO8 output
52	VLDO9	LDO9 output
53	VCC9	Power supply for LDO
54	VSWOUT	Switch output
55	VREF	Internal reference voltage
56	REFGND	Reference ground
57	SNSN	Bat charging and discharging sense current negative pin
58	SNSP	Bat charging and discharging sense current positive pin
59,60	BAT	Positive battery terminal
61,62	SYS	DC-DC regulator output to power the system load and charge the battery
63	VCC4	Power supply for DCDC4
64	SW4	Switch output of DCDC4
65	VFB4	DCDC4 output voltage feedback input
66	GND4	Power ground for DCDC4
67	TS1	Thermistor1 input. Connect a thermistor from this pin to ground. The thermistor is usually inside the battery pack.
68	TS2	Thermistor2 input. Connect a thermistor from this pin to ground. Or it can be used as analog input pin of internal ADC if the control bit is set to ADC function.
Exposed pad	Exposed ground	It must be connected to ground for thermal and electrical enhancement.

1.4 Key Concepts

- I2C Address

7-bit slave address: 0x1c

- The PMIC has three operating modes

1. PMIC Normal Mode

When the system is running normally, the PMIC operates in normal mode, with the pmic_sleep pin held low.

2. PMIC Sleep Mode

To minimize standby power consumption during system suspend, the PMIC switches to sleep mode. In this state, output voltages for certain rails are typically reduced or outputs are disabled entirely, configurable based on specific product requirements. During system standby, the AP configures the pmic_sleep mode to sleep via an I2C command and then drives the pmic_sleep pin high to enter sleep state. Upon SoC wakeup, the pmic_sleep pin returns to low, causing the PMIC to exit sleep mode.

3. PMIC Shutdown Mode

When the system initiates the shutdown sequence, the PMIC must perform power-down operations for the entire system. The AP configures the pmic_sleep mode to shutdown via an I2C command and then drives the pmic_sleep pin high to enter shutdown state.

- pmic_sleep Pin

Normally held low, indicating the PMIC is in normal mode. When driven high, the PMIC switches to either sleep or shutdown mode.

- pmic_int Pin

Normally held high. When an Interrupt occurs, the pin goes low. If the Interrupt remains unhandled, the pin stays low.

- pmic_pwron Pin

The pwrkey function requires the hardware power button to be connected to this pin. The Driver uses this pin to detect button press and release events.

- Operating Modes of DCDC Rails

  The DCDC supports PWM (also known as force PWM) and PFM modes. However, the PMIC features a mode that dynamically switches between PWM and PFM, commonly referred to as AUTO mode. The PMIC supports two operating modes: PWM and AUTO PWM/PFM. While AUTO mode offers higher efficiency, it exhibits poorer ripple and transient response. For system stability, the device is configured to operate in PWM mode during runtime and switches to AUTO PWM/PFM mode when the system enters sleep.

- DCDC3 Voltage Regulation

The DCDC3 power rail is unique; its voltage cannot be modified via registers and must be adjusted using external voltage divider resistors. Therefore, to change the voltage, the peripheral hardware must be modified. In Rockchip solutions, it is typically used as VCC_DDR.

- Runtime Voltage Regulation Range for DCDC and LDO

1. Continuous DCDC Voltage Range:

Voltage Range (V)	Step Value (mV)	Specific Voltage Levels (V)
0.7125 ~ 1.45	12.5	0.7125, 0.725, 0.7375, ..., 1.45
1.8 ~ 3.3	100	1.8, 1.9, 2.0, 2.2, ..., 3.3

2. Continuous LDO Voltage Range:

Voltage Range (V)	Step Value (mV)	Specific Voltage Levels (V)
0.8 ~ 3.4	100	0.8, 0.9, 1.0, 1.1, 1.2, ... 3.4

1.5 Power-On Conditions and Sequence

1. Power-On Conditions

The PMIC can be powered on if any of the following conditions are met:

- The EN signal transitions from low to high.
- The EN signal remains high, and an RTC alarm Interrupt is triggered.
- The EN signal remains high, and the PWRON button is pressed.

2. Power-On Sequence

Power-on sequence requirements for each power rail may vary across different SoC platforms. The current power-on sequences are listed below; please refer to the latest datasheet for specific details:

13 上电启动时序（POWER SEQUENCE）

AP	RK3188/RK3168/ RK3188M/RK3168M/ RK3028A/RK3028 /RK2928		部分 otp/BUCK1~4,LDO3/LDO4 / LDO5/LDO7		RK3066		RK3288/RK3368		S-Product	
BOOT	11		10		01		00 RK818-1		RK818-2	
	电压默认值	上电时序	电压默认值	上电时序	电压默认值	上电时序	电压默认值	上电时序	电压默认值	上电时序
BUCK1	1.1V	3	OTP	OTP	1.2V	3	1.1V	3	1.0V	12
BUCK2	1.1V	1	OTP	OTP	1.2V	1	1.1V	1	1.0V	12
BUCK3	x	4	x	OTP	x	4	X	3	X	13
BUCK4	3.0V	1	OTP	OTP	3.0V	1	3.3V	4	3.3V	14
LDO1	3.3V	x	3.3V	x	3.3V	x	3.3V	x	1.8V	11
LDO2	3.0V	x	3V	x	3.0V	x	3.0V	x	X	X
LDO3	1.1V	1	OTP	OTP	1.1V	1	1.1V	x	1.8V	15
LDO4	2.5V	2	OTP	OTP	2.5V	2	2.5V	x	1.8V	1
LDO5	3V	1	OTP	OTP	3.0V	2	1.8V	4	1.8V	11
LDO6	1.2V	x	1.2V	x	1.1V	x	1.1V	x	X	X
LDO7	1.8V	2	OTP	OTP	1.8V	2	1.8V	3	1.1V	15
LDO8	1.8V	x	1.8V	x	1.8V	x	1.8V	x	3.0V	14
LDO9	3.0V	4	3.0V	5	3.0V	4	3.3V	10	1.8V	15
SWITCH	x	x	x	x	x	x	x	10	x	x
OTG	5V	x	5V	x	5V	x	5V	x	5V	x
HDMI_5V	5V	x	5V	x	5V	x	5V	x	5V	x

2. Configuration

2.1 Driver and menuconfig

Kernel 3.10 Configuration

RK818 Driver file:

```
drivers/mfd/rk818.c
drivers/mfd/rk818-irq.c
drivers/rtc/rtc-rk818.c
drivers/power/rk818-battery.c
```

For the RK818 DTS file, refer to:

```
arch/arm/boot/dts/rk818.dtsi
arch/arm64/boot/dts/rk3368-p9_818.dts
```

Macro configuration in menuconfig:

```
CONFIG_MFD_RK818
CONFIG_RTC_RK818
CONFIG_BATTERY_RK818
```

4.4 Kernel Configuration

RK818 Driver file:

```
drivers/mfd/rk808.c
drivers/rtc/rtc-rk808.c
drivers/regulator/rk808-regulator.c
drivers/clk/clk-rk808.c
drivers/power/rk818_charger.c
drivers/power/rk818_battery.c
```

Macro configuration corresponding to menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_REGULATOR_RK808
CONFIG_BATTERY_RK818
CONFIG_CHARGER_RK818
CONFIG_COMMON_CLK_RK808
```

4.19 Kernel Configuration

RK818 Driver file:

```
drivers/mfd/rk808.c
drivers/rtc/rtc-rk808.c
drivers/regulator/rk808-regulator.c     // Different from the 4.4 kernel
drivers/clk/clk-rk808.c
drivers/power/supply/rk818_battery.c
drivers/power/supply/rk818_charger.c
```

Macro configuration corresponding to menuconfig:

```
CONFIG_MFD_RK808
CONFIG_RTC_RK808
CONFIG_REGULATOR_RK808
CONFIG_BATTERY_RK818
CONFIG_CHARGER_RK818
CONFIG_COMMON_CLK_RK808
```

2.2 DTS Configuration

3.10 Kernel DTS Configuration

The DTS configuration includes: I2C bus attachment, main device, regulator, RTC, poweroff, and other sections.

```
&i2c1 {
    rk818: rk818@1c {
        reg = <0x1c>;
        status = "okay";
    };
};

/include/ "../../../arm/boot/dts/rk818.dtsi"
&rk818 {
    gpios =<&gpio0 GPIO_A1 GPIO_ACTIVE_HIGH>,<&gpio0 GPIO_A0 GPIO_ACTIVE_LOW>;
    rk818,system-power-controller;
    pinctrl-names = "default";
    pinctrl-0 = <&gpio0_c1>;

    regulators {
        rk818_dcdc1_reg: regulator@0{
            regulator-name= "vdd_arm";/*vcc arm*/
            regulator-min-microvolt = <700000>;/*<725000>;*/
            regulator-max-microvolt = <1500000>;
            regulator-initial-mode = <0x2>;
            regulator-initial-state = <3>;
            regulator-state-mem {
                regulator-state-mode = <0x2>;
                regulator-state-disabled;
                regulator-state-uv =<900000>;
            };
        };

        rk818_dcdc2_reg: regulator@1 {
            regulator-name= "vdd_logic";/*vcc gpu*/
            regulator-min-microvolt = <700000>;
            regulator-max-microvolt = <1200000>;
            regulator-initial-mode = <0x2>;
            regulator-initial-state = <3>;
            regulator-state-mem {
```

```
                regulator-state-mode = <0x2>;
                regulator-state-enabled;
                regulator-state-uv = <1200000>;
            };
        };
        rk818_dcdc3_reg: regulator@2 {
                    .............
        };
        ..............................
    };
};
```

1. I2C Attachment

The complete rk818 node must be attached under the corresponding i2c node, with status = "okay"; configured.

2. Main Section

- Non-modifiable Parts

```
rk818,system-power-controller: Declare that the RK818 supports system power-down management.
```

- Modifiable sections

gpios: Specify the pmic_int (first) and pmic_sleep (second) pins;

3. Regulator section

- `regulator-name`: Power supply name; it is recommended to keep it consistent with the hardware schematic, as this name must match when using the regulator_get interface;
- `regulator-min-microvolt`: Minimum voltage adjustable at runtime;
- `regulator-max-microvolt`: Maximum voltage adjustable at runtime;
- `regulator-initial-mode`: DCDC operating mode at runtime, generally configured as 1. 1: force PWM, 2: auto PWM/PFM;
- `regulator-state-mode`: DCDC operating mode during suspend, generally configured as 2. 1: force PWM, 2: auto PWM/PFM;
- `regulator-initial-state`: Mode during suspend; must be configured as 3;
- `regulator-boot-on`: When this property is present, the power supply is enabled upon regulator registration;
- `regulator-always-on`: When this property is present, the power supply cannot be disabled at runtime and is enabled upon registration;
- `regulator-state-enabled`: Maintains power-on state during suspend; to disable this power supply during suspend, change to "regulator-state-disabled";
- `regulator-state-uv`: Standby voltage when power is maintained during suspend.

Note:

If regulator-min-microvolt and regulator-max-microvolt are set to the same voltage, the system framework will automatically set this voltage and enable the power supply upon regulator registration, requiring no user intervention.

```
If `regulator-boot-on` or `regulator-always-on` is present, the system framework will enable the regulator by default during registration. In this case, the regulator voltage depends on two scenarios: if `regulator-min-microvolt` and `regulator-max-microvolt` are equal, the system framework sets the voltage to this specific value; if `regulator-min-microvolt` and `regulator-max-microvolt` differ, the voltage defaults to the PMIC's hardware default power-on voltage.
```

4. RTC Section

If the RTC function does not need to be enabled (e.g., on box products), add the node as described above and explicitly set `status = "disabled"`. If enabling is required, either remove the entire RTC node or set its status to `status = "okay"`.

5. Poweroff Section

The RK818 Driver automatically intercepts shutdown commands and executes I2C writes to disable PMIC outputs.

`rk818_shutdown` is registered as a syscore shutdown handler to perform preparatory tasks, such as disabling RTC Interrupts and other specific operations.

```
static void rk818_shutdown(void)
{
        int ret;
        struct rk818 *rk818 = g_rk818;

        pr_info("%s\n", __func__);
        ret = rk818_set_bits(rk818, RK818_INT_STS_MSK_REG1,(0x3<<5),(0x3<<5)); //close rtc
int when power off
        ret = rk818_clear_bits(rk818, RK818_RTC_INT_REG,(0x3<<2)); //close rtc int when power
off
        /*disable otg_en*/
        ret = rk818_clear_bits(rk818, RK818_DCDC_EN_REG, (0x1<<7));

        mutex_lock(&rk818->io_lock);
        mdelay(100);
}

static struct syscore_ops rk818_syscore_ops = {
        .shutdown = rk818_shutdown,
};
```

rk818_device_shutdown actually writes to the I2C bus to disable the PMIC outputs.

```
void rk818_device_shutdown(void)
{
        int ret, i;
        u8 reg = 0;
        struct rk818 *rk818 = g_rk818;

        for (i = 0; i < 10; i++) {
                pr_info("%s\n", __func__);
```

```
                ret = rk818_i2c_read(rk818, RK818_DEVCTRL_REG, 1, &reg);
                if (ret < 0)
                        continue;
                ret = rk818_i2c_write(rk818, RK818_DEVCTRL_REG, 1,
                                      (reg | (0x1 << 0)));
                if (ret < 0) {
                        pr_err("rk818 power off error!\n");
                        continue;
                }
        }
        while(1) wfi();
}
EXPORT_SYMBOL_GPL(rk818_device_shutdown);
```

4.4 Kernel DTS Configuration

DTS configuration includes: I2C bus attachment, main device, RTC, CLK, regulator, charger, battery, and other sections.

```
&i2c1 {
        status = "okay";
        rk818: pmic@1c {
                compatible = "rockchip,rk818";
                reg = <0x1c>;
                status = "okay";

                clock-output-names = "rk818-clkout1", "wifibt_32kin";
                interrupt-parent = <&gpio0>;
                interrupts = <1 IRQ_TYPE_LEVEL_LOW>;
                pinctrl-names = "default";
                pinctrl-0 = <&pmic_int_l>;
                rockchip,system-power-controller;
                wakeup-source;
                #clock-cells = <1>;

                vcc1-supply = <&vcc_sys>;
                vcc2-supply = <&vcc_sys>;
                vcc3-supply = <&vcc_sys>;
                vcc4-supply = <&vcc_sys>;
                vcc6-supply = <&vcc_sys>;
                vcc7-supply = <&vcc_sys>;
                vcc8-supply = <&vcc_sys>;
                vcc9-supply = <&vcc_io>;

                regulators {
                        vdd_logic: DCDC_REG1 {
                                regulator-name = "vdd_logic";
                                regulator-always-on;
                                regulator-boot-on;
                                regulator-min-microvolt = <750000>;
                                regulator-max-microvolt = <1450000>;
                                regulator-ramp-delay = <6001>;
                                regulator-state-mem {
```

```
                                        regulator-on-in-suspend;
                                        regulator-suspend-microvolt = <1000000>;
                                };
                        };

                        vdd_gpu: DCDC_REG2 {
                                regulator-name = "vdd_gpu";
                                regulator-always-on;
                                regulator-boot-on;
                                regulator-min-microvolt = <800000>;
                                regulator-max-microvolt = <1250000>;
                                regulator-ramp-delay = <6001>;
                                regulator-state-mem {
                                        regulator-on-in-suspend;
                                        regulator-suspend-microvolt = <1000000>;
                                };
                        };
                        vcc_ddr: RK818_DCDC3@2 {
                                ................
                        };
                        ............................
                };
        };
};
```

1. I2C Attachment

The complete RK818 node must be attached under the corresponding I2C node, with `status = "okay";` configured.

2. Main Section

- Do not modify:

```
compatible = "rockchip,rk818";
reg = <0x1c>;
rockchip,system-power-controller;
wakeup-source;
#clock-cells = <1>;
```

- Modifiable (according to pinctrl rules)

interrupt-parent: Specifies which GPIO the pmic_int belongs to; interrupts: Specifies the pin index and polarity of pmic_int on the GPIO defined in interrupt-parent; pinctrl-names: Do not modify; fixed as "default"; pinctrl-0: References the pmic_int pin defined in pinctrl;

3. rtc

If this module is selected in menuconfig but the drivers do not need to be enabled in practice, you can add an rtc node in the DTS and explicitly set its status to status = "disabled". This prevents the drivers from being enabled, although error logs will appear during boot and can be ignored. To enable the drivers, either remove the corresponding node or set the status to status = "okay".

4. regulator

- `regulator-compatible`: The name used for matching during Driver registration. Do not modify this value, or the Driver will fail to load.
- `regulator-name`: The power supply name. It is recommended to keep this consistent with the hardware schematic, as the `regulator_get` interface requires a match with this name.
- `regulator-init-microvolt`: The initialization voltage during the U-Boot stage; this is invalid in the kernel stage.
- `regulator-min-microvolt`: The minimum voltage adjustable during runtime.
- `regulator-max-microvolt`: The maximum voltage adjustable during runtime.
- `regulator-initial-mode`: The DCDC operating mode during runtime, typically configured as 1. Values: 1: force PWM, 2: auto PWM/PFM.
- `regulator-mode`: The DCDC operating mode during sleep, typically configured as 2. Values: 1: force PWM, 2: auto PWM/PFM.
- `regulator-initial-state`: The mode during suspend; must be configured as 3.
- `regulator-boot-on`: If this property exists, the power supply is enabled upon Driver registration.
- `regulator-always-on`: If this property exists, the power supply cannot be disabled during runtime and is enabled upon Driver registration.
- `regulator-ramp-delay`: The DCDC voltage rise time; fixed at 12500.
- `regulator-on-in-suspend`: Maintains power status during sleep. To disable this power supply during sleep, change this to `regulator-off-in-suspend`.
- `regulator-suspend-microvolt`: The standby voltage when power is maintained during sleep.

5. Power-off Section

On kernel 4.4, `pm_power_off_prepare` is used to implement pre-shutdown preparations for the PMIC, such as disabling RTC interrupts and configuring specific registers. Register `syscore_shutdown` to perform the actual PMIC shutdown.

6. Clock Section

If a node needs to reference the RK818 clock, use the following format:

`clocks = <&rk818 1>;` First parameter: `&rk818` is fixed and must not be modified. Second parameter: Specifies which RK818 clock to reference; valid values are 0 or 1, where 0 corresponds to `rk818-clkout1` and 1 corresponds to `rk818-clkout2`.

Kernel 4.19 DTS Configuration

Refer to the kernel 4.4 DTS configuration. The difference is that the kernel 4.19 DTS configuration no longer requires the gpio subnode, but other modules still reference and use RK818 pins using the format `gpios = <&rk818 0 GPIO_ACTIVE_LOW>;`.

2.3 Function Interfaces

The following interfaces generally meet daily usage requirements, including regulator enable/disable, voltage setting, and voltage retrieval:

1. Obtain regulator:

```
struct regulator *regulator_get(struct device *dev, const char *id)
```

Set `dev` to NULL by default; `id` corresponds to the `regulator-name` property in the DTS.

2. Release regulator

`void regulator_put(struct regulator *regulator)`

3. Enable regulator

`int regulator_enable(struct regulator *regulator)`

4. Disable regulator

`int regulator_disable(struct regulator *regulator)`

5. Obtain regulator voltage

`int regulator_get_voltage(struct regulator *regulator)`

6. Set regulator voltage

`int regulator_set_voltage(struct regulator *regulator, int min_uV, int max_uV)`

The caller must ensure that min_uV equals max_uV when passing parameters.

7. Example

```
struct regulator *rdev_logic;

rdev_logic = regulator_get(NULL, "vdd_logic");                  // Get vdd_logic
regulator_enable(rdev_logic);                                                  // Enable
vdd_logic
regulator_set_voltage(rdev_logic, 1100000, 1100000);    // Set voltage to 1.1V
regulator_disable(rdev_logic);                                                 // Disable
vdd_logic
regulator_put(rdev_logic);                                                            //
Release vdd_logic
```

Note: Kernels 4.4 and 4.19 also provide `devm_`-prefixed regulator interfaces to help developers manage requested resources.

3. Debug

3.1 Kernel 3.10

Since the logic of drivers involving the PMIC is not complex, the focus lies on the final register configurations. Therefore, the common debug method is to directly read RK818 registers via the following node:

`/sys/rk818/rk818_test`

Read register:

`echo r [addr] > /sys/rk818/rk818_test`

Write register:

```
echo w [addr] [value] > /sys/rk818/rk818_test
```

3.2 Kernel 4.4

The command format is the same as in kernel 3.10, but the node path differs. The debug node path for kernel 4.4 is:

```
/sys/rk8xx/rk8xx_dbg
```

3.3 Kernel 4.19

Refer to the commands for kernel 4.4.